Q1. Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

Answer:

Consider the following query:

```
SELECT S.Name, C.CourseName
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Course C ON E.CourseID = C.CourseID
WHERE S.Department = 'CSE'
AND C.Credits > 3
```

Initial relational algebra expression

The initial query can be represented as:

```
π S.Name, C.CourseName
      |
σ S.Department='CSE' AND C.Credits>3
      |
      ⋈
     / \
    ⋈   Course
   / \
Student Enrollment
```

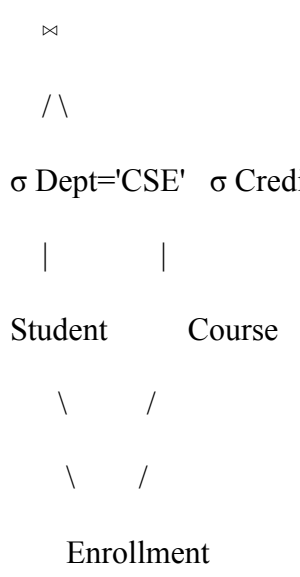
Step 1: Push selections down

Instead of applying the selection after all joins, apply each condition directly to its relation:

σ Department='CSE'(Student) and σ Credits>3(Course)

This reduces the number of tuples participating in joins.

Step 2: Perform the most selective operations first



Step 3: Push projections down

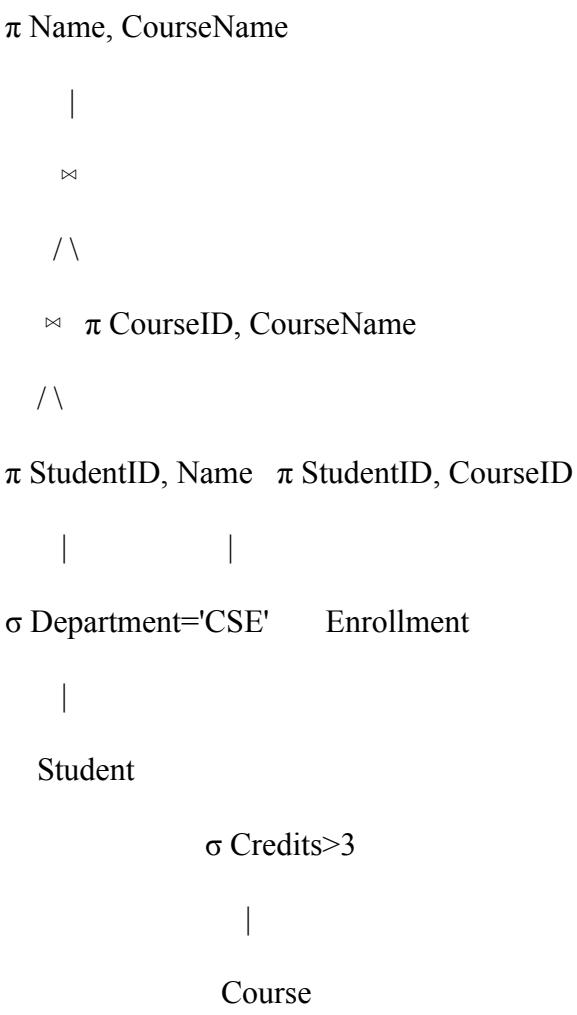
Only required attributes are retained:

Student → {StudentID, Name}

Enrollment → {StudentID, CourseID}

Course → {CourseID, CourseName}

Optimized execution plan



Conclusion

The heuristic optimization reduces the size of intermediate relations by:

1. Pushing selections down.
2. Performing selective operations early.
3. Pushing projections down.
4. Performing joins after reducing the input relations.

This improves query execution time and reduces memory and disk I/O.

Q2. Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join.

Answer:

Assume:

Relation	Tuples	Pages
R	1,000	100
S	10,000	500

Assume the join condition is:

R.A = S.A

1. Nested-loop join

If R is the outer relation:

Cost ≈ B(R) + B(R) × B(S)

= 100 + (100 × 500)

= 50,100 page I/Os

Therefore, nested-loop join has a high cost.

2. Sort-merge join

Both relations need to be sorted before merging.

Approximate external sorting cost:

Sort(R) + Sort(S) + Merge

Because both relations are relatively large, sorting requires several disk I/Os.

3. Hash join

For an equality join:

R.A = S.A

hash join is usually highly efficient.

Approximate cost for a two-pass hash join:

3(B(R) + B(S))

= 3(100 + 500)

= 1,800 page I/Os

Comparison

Join Algorithm	Approximate Cost	Suitability
Nested-loop	50,100 I/Os	Poor
Sort-merge	Higher than hash in this case	Moderate
Hash join	~1,800 I/Os	Best

Conclusion

Hash join is selected because the join is an **equality join** and it has the lowest estimated I/O cost among the alternatives.

Q3. Implement JDBC connectivity in Java to a MySQL database and execute parameterized queries using PreparedStatement.

Answer:

JDBC allows a Java application to communicate with a MySQL database.

Java Program

```
import java.sql.*;

public class JDBCExample {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/college";

        String user = "root";

        String password = "";

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(

                url, user, password

            );

            String sql = "SELECT * FROM Student WHERE Department = ?";

            PreparedStatement ps = con.prepareStatement(sql);

            ps.setString(1, "CSE");

            ResultSet rs = ps.executeQuery();

            while (rs.next()) {

                System.out.println(

                    rs.getInt("StudentID") + " " +

                    rs.getString("Name") + " " +
```

```
                rs.getString("Department")
            );
        }
        rs.close();
        ps.close();
        con.close();
    } catch (Exception e) {
        System.out.println(e);
    }
  }
}
```

Explanation

1. Class.forName() loads the MySQL JDBC driver.
2. DriverManager.getConnection() establishes the connection.
3. PreparedStatement creates a parameterized SQL query.
4. ? represents a parameter.
5. setString() supplies the parameter value.
6. executeQuery() executes the SELECT statement.
7. ResultSet stores the returned records.
8. The connection is closed after execution.

Advantages of PreparedStatement

- Prevents SQL injection.
- Improves security.
- Supports parameterized queries.
- Can improve performance for repeated queries.

Q4. Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.

Answer:

Consider the following schedule:

T1: R(A) W(A)

T2: R(A) W(B)

T3: R(B)

T4: R(A)

A **precedence graph** contains:

- One node for each transaction.
- An edge Ti → Tj when an operation of Ti conflicts with and occurs before an operation of Tj.

Conflict rules

Conflicts occur when:

- Both operations access the same data item.
- At least one operation is a write.

Precedence graph

Suppose the schedule produces:

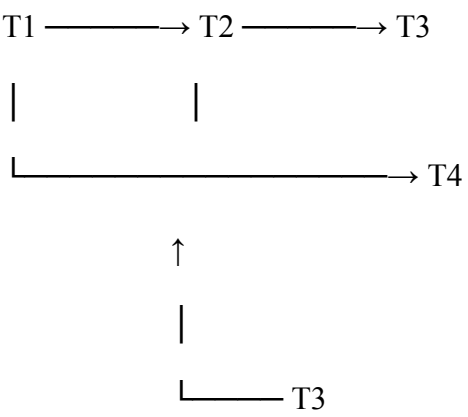
T1 → T2

T1 → T4

T2 → T3

T3 → T4

Graph:



There is **no cycle**.

Rule

No cycle → Conflict Serializable

Cycle → Not Conflict Serializable

Therefore, since the precedence graph is acyclic, the schedule is **conflict-serializable**.

A possible serial order is:

T1 → T2 → T3 → T4

Q5. Demonstrate Basic Two-Phase Locking (2PL) and identify and resolve deadlocks.

Answer:

Two-Phase Locking divides a transaction into two phases:

1. Growing phase

A transaction can:

- Acquire locks.
- Cannot release locks.

2. Shrinking phase

A transaction can:

- Release locks.
- Cannot acquire new locks.

Example

Suppose:

T1: Lock-X(A)

Lock-X(B)

Write(A)

Write(B)

Unlock(A)

Unlock(B)

T2: Lock-X(B)

Lock-X(A)

Write(B)

Write(A)

Unlock(B)

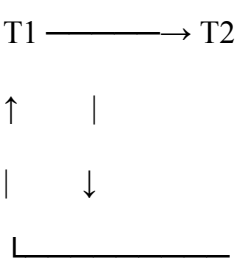
Unlock(A)

A deadlock can occur:

T1 holds A → waiting for B

T2 holds B → waiting for A

Graph:



There is a cycle:

T1 → T2 → T1

Therefore, a deadlock exists.

Deadlock resolution

One transaction can be aborted:

Abort T2

↓

Release B

↓

T1 obtains B

↓

T1 completes

↓

T2 restarts

Conclusion

Basic 2PL guarantees conflict serializability, but **deadlocks can occur**. Deadlocks can be handled using detection and recovery or prevention techniques.

Q6. Apply Wait-Die and Wound-Wait timestamp-based deadlock prevention.

Answer:

Assume:

T1 timestamp = 10 → Older

T2 timestamp = 20 → Younger

T1 requests a lock held by T2.

Wait-Die

Rule:

Older requests younger → Older waits

Younger requests older → Younger dies/aborts

Here:

T1 (older) → requests lock held by T2 (younger)

Therefore:

T1 waits

T2 continues

If T2 requests a lock held by T1:

T2 (younger) → requests T1's lock

Therefore:

T2 aborts

Wound-Wait

Rule:

Older requests younger → Younger is aborted

Younger requests older → Younger waits

Therefore, when T1 requests T2's lock:

T1 (older)

↓

T2 (younger)

↓

T2 is aborted

If T2 requests T1's lock:

T2 waits for T1

Comparison

Situation	Wait-Die	Wound-Wait
Older → Younger	Older waits	Younger aborts
Younger → Older	Younger aborts	Younger waits

Both techniques prevent circular waiting and therefore prevent deadlocks.

Q7. Given transaction logs with checkpoints, apply Immediate Update recovery after system failure.

Answer:

In **Immediate Update**, database modifications may be written to disk before the transaction commits.

Therefore, after a crash:

- Committed transactions may need **REDO**.
- Uncommitted transactions need **UNDO**.

Consider:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 250>

<COMMIT T1>

<CHECKPOINT>

<T2, C, 300, 350>

System Crash

Step 1: Identify committed transactions

T1 → COMMITTED

T2 → UNCOMMITTED

Step 2: REDO committed transactions

T1 has committed:

A: 100 → 150

Therefore:

REDO T1

Step 3: UNDO uncommitted transactions

T2 has not committed:

B: 200 → 250

C: 300 → 350

Therefore:

UNDO T2

Restore:

B = 200

C = 300

Recovery result

T1 → REDO

T2 → UNDO

The checkpoint helps reduce the amount of the log that must be examined during recovery.

Q8. Apply Deferred Update (NO-UNDO/REDO) recovery on a log file.

Answer:

In **Deferred Update**, database changes are not written to the database until the transaction commits.

Therefore:

Committed transaction → REDO

Uncommitted transaction → Ignore

Consider:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 250>

<COMMIT T1>

<T2, C, 300, 350>

CRASH

Step 1: Find committed transactions

T1 → committed

T2 → uncommitted

Step 2: REDO T1

A = 150

Step 3: Ignore T2

Since T2 did not commit:

B remains 200

C remains 300

Recovery table

Transaction	Status	Recovery
T1	Committed	REDO
T2	Uncommitted	Ignore

Thus, Deferred Update follows the **NO-UNDO/REDO** recovery technique.

Q9. Design a concurrency-control mechanism using Strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.

Answer:

Consider two bank accounts:

A = ₹10,000

B = ₹5,000

T1 transfers ₹2,000 from A to B.

T2 reads the balance of A and B.

T1 – Transfer

LOCK-X(A)

LOCK-X(B)

A = A - 2000

B = B + 2000

COMMIT

UNLOCK(A)

UNLOCK(B)

After transfer:

A = ₹8,000

B = ₹7,000

T2 – Read balance

LOCK-S(A)

LOCK-S(B)

Read A

Read B

COMMIT

UNLOCK(A)

UNLOCK(B)

Strict 2PL

In Strict 2PL:

- Exclusive locks are held until commit/abort.
- Other transactions cannot read uncommitted changes.
- This prevents dirty reads.

Concurrent execution

T1: LOCK-X(A)

T1: LOCK-X(B)

T1: Update A and B

T1: COMMIT

T1: UNLOCK(A,B)

T2: LOCK-S(A)

T2: LOCK-S(B)

T2: Read balances

T2: COMMIT

T2 therefore sees a consistent state:

A = ₹8,000

B = ₹7,000

Benefits

Strict 2PL provides:

- Conflict serializability.
- No dirty reads.
- No cascading rollback.
- Consistent banking transactions.

Q10. Write pseudocode for query optimization using heuristics.

Answer:

The three major heuristics specified in the assignment are:

1. Push selections down.
2. Project early.
3. Order joins by smaller intermediate results.

Pseudocode

Algorithm HeuristicQueryOptimization(Query)

BEGIN

Parse Query

Convert SQL query into relational algebra

// Step 1: Push selections down

FOR each selection condition

Move selection as close as possible

to the base relation

END FOR

```
    // Step 2: Project early
    FOR each relation
        Keep only attributes required for
        selection, join and final output
    END FOR
    // Step 3: Order joins
    Estimate size of intermediate relations
    Select the smallest relation first
    FOR remaining relations
        Choose the join that produces
        the smallest intermediate result
    END FOR
    Generate optimized execution plan
    RETURN optimized execution plan
END
```

Example

Original:

```
π
|
σ
|
```

⋈

/ \

R ⋈

/ \

S T

After heuristic optimization:

π

|

⋈

/ \

σ(R) ⋈

/ \

σ(S) σ(T)

The selections are performed early, unnecessary columns are removed using projection, and joins are arranged to minimize intermediate results.

Conclusion

Heuristic optimization improves query performance without calculating the exact cost of every possible execution plan. It reduces:

- Intermediate relation size.
- Disk I/O.
- Memory usage.
- Overall query execution time.